Java w IntelliJ IDEA - pętle i rekurencja

BLOK 4 - wielokrotne wykonywanie instrukcji

1. Co to jest iteracja?

Gdy potrzebujemy wykonać tę samą operację kilkanaście razy, nie warto ręcznie przepisywać identycznych instrukcji. Możemy użyć **pętli**, czyli konstrukcji wielokrotnie wykonującej fragment kodu. Jedno wykonanie jej wnętrza nazywamy **iteracją**. Liczba iteracji zależy od warunku oraz od tego, jak w czasie wykonywania zmieniają się wykorzystywane wartości.

```
int counter = 1;
while (counter <= 3) {
    System.out.println("Numer: " + counter);
    counter++;
}
```

```
Wynik:
Numer: 1
Numer: 2
Numer: 3
```

Na początku `counter` ma wartość 1. Warunek `counter <= 3` jest prawdziwy, więc program wyświetla 1 i zwiększa licznik do 2. Następnie ponownie sprawdza warunek. Dla licznika 2 wykonuje blok drugi raz, dla 3 trzeci, a dla 4 warunek staje się fałszywy. Dzięki zwiększaniu licznika pętla potrafi się zakończyć.

2. Pętla while - warunek sprawdzany przed wejściem do bloku

Pętla `while` najpierw sprawdza warunek. Jeżeli od razu otrzyma `false`, jej wnętrze nie wykona się ani razu. To wygodne, gdy liczba powtórzeń jest nieznana, a dalsza praca zależy od jakiegoś warunku, na przykład od tego, czy użytkownik wprowadza dane.

```
int fuel = 0;
while (fuel > 0) {
    System.out.println("Silnik pracuje");
    fuel--;
}
System.out.println("Koniec");
```

```
Wynik:
Koniec
```

Pętla nie wyświetli „Silnik pracuje”, bo `fuel > 0` jest od razu fałszywe. Właśnie dlatego `while` jest dobrą konstrukcją, gdy pewna czynność nie powinna zostać wykonana bez spełnienia wymaganego warunku. Aby pętla nie trwała bez końca, jej kod musi zmieniać dane albo stan prowadzący do zakończenia.

3. Pętla do-while - najpierw działanie, potem sprawdzenie

W pętli `do-while` kod wykonuje się przynajmniej jeden raz. Dopiero po wykonaniu bloku program sprawdza warunek i decyduje o kolejnym powtórzeniu. Ten sposób przydaje się np. w prostym menu, które powinno pojawić się przynajmniej raz, zanim zdecydujemy o dalszym działaniu.

```
int number = 5;
do {
    System.out.println(number);
    number++;
} while (number < 5);
```

```
Wynik:
5
```

Mimo że `number < 5` jest fałszywe, program wypisze 5, ponieważ warunek zostaje sprawdzony dopiero po pierwszym wykonaniu bloku. Dla porównania analogiczne `while (number < 5)` nie wypisałoby niczego. Zwróć uwagę na średnik po nawiasie warunku - w składni `do-while` jest wymagany.

Pętla	**Kiedy sprawdza warunek?**	**Czy wykona blok co najmniej raz?**
`while`	**przed wykonaniem bloku**	**nie zawsze**
`do-while`	po wykonaniu bloku	tak
`for`	przed kolejnymi przebiegami	nie zawsze

4. Pętla for - początek, warunek i krok w jednym miejscu

Pętla `for` jest często stosowana, gdy potrafimy opisać początkową wartość licznika, warunek kontynuowania i sposób zmiany licznika. Jej nagłówek ma postać `for (inicjalizacja; warunek; krok)`. Zazwyczaj po każdej iteracji wykonywany jest krok, a następnie ponownie sprawdzany warunek.

```
for (int i = 0; i < 4; i++) {
    System.out.println(i);
}
```

```
Wynik:
0
1
2
3
```

Licznik zaczyna od 0, a warunek mówi `i < 4`, dlatego ostatnią wypisaną liczbą jest 3. Gdy `i` osiągnie 4, blok nie zostaje już wykonany. Jeżeli polecenie brzmi „wyświetl liczby od 1 do 4”, potrzebujemy innego początku: `int i = 1`, a warunek `i <= 4`. Wiele pomyłek w pętlach

jest skutkiem złego wyboru granicy.

5. Licznik i akumulator to nie to samo

Licznik mówi zwykle, którą iterację wykonujemy. Akumulator przechowuje wynik narastający w kolejnych iteracjach, na przykład sumę liczb. Używanie dwóch osobnych zmiennych pomaga rozumieć algorytm: jedna steruje przebiegiem, druga zbiera rezultat.

```
int sum = 0;
for (int i = 1; i <= 4; i++) {
    sum = sum + i;
}
System.out.println(sum);
```

```
Wynik:
10
```

Po pierwszej iteracji `sum` wynosi 1, po drugiej 3, po trzeciej 6, a po czwartej 10. Warto rozpisać te wartości ręcznie w tabelce: `i = 1, sum = 1`; `i = 2, sum = 3`; `i = 3, sum = 6`; `i = 4, sum = 10`. Właśnie taka analiza pomaga przy błędach polegających na użyciu niewłaściwej zmiennej.

6. Pętla nieskończona - dlaczego jest niebezpieczna?

Jeśli warunek pętli pozostaje prawdziwy i nie dochodzi do instrukcji kończącej, pętla może działać bez końca. Na przykład `while (true)` tworzy pętlę bezwarunkową. Czasem jest to celowe, np. w programie czekającym na zdarzenia, ale na początku łatwo stworzyć ją przez pomyłkę.

```
int value = 1;
while (value <= 3) {
    System.out.println(value);
    value++;  // bez tej linii licznik nie ulegałby zmianie
}
```

Przy analizie podejrzanej pętli zadaj sobie trzy pytania: jaka jest wartość przed pierwszym wejściem? Co zmienia się w bloku? Kiedy warunek stanie się fałszywy? Jeżeli nie umiesz wskazać możliwej chwili zakończenia, poszukaj brakującej zmiany licznika lub niepoprawnego warunku.

7. break - przerwij wykonywanie pętli

`break` kończy najbliższą pętlę, w której się znajduje, i przenosi wykonanie do instrukcji za tą pętlą. Przydaje się, gdy znajdziemy poszukiwaną wartość albo wykryjemy sytuację, w której nie potrzebujemy dalszego sprawdzania.

```
for (int i = 1; i <= 6; i++) {
    if (i == 4) {
        break;
    }
    System.out.println(i);
}
System.out.println("Po pętli");
```

```
Wynik:
1
2
3
Po pętli
```

Dla `i = 4` warunek w `if` jest prawdziwy i wykonywane jest `break`. Liczba 4 nie zostaje wypisana, ponieważ instrukcja wyświetlania znajduje się po `break`. Pętla nie przechodzi już do wartości 5 i 6.

8. continue - pomiń jedną iterację

`continue` nie kończy całej pętli. Przerywa jedynie aktualną iterację i przechodzi do kolejnego kroku pętli. W `for` wykonywany jest wtedy krok licznika, a później kontrola warunku.

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        continue;
    }
    System.out.println(i);
}
```

```
Wynik:
1
2
4
5
```

Wartość 3 zostaje pominięta, ale wartości 4 i 5 nadal pojawiają się na ekranie. Porównaj tę sytuację z `break`, które zakończyłoby całą pętlę przy 3. W zagnieżdżonych pętlach oba polecenia domyślnie dotyczą najbliższej pętli, dlatego znaczenie ma ich położenie.

9. Pętle zagnieżdżone - jedna pętla wewnątrz drugiej

Pętla może zawierać inną pętlę. Taka konstrukcja jest potrzebna np. przy tabelach, siatkach czy tablicach dwuwymiarowych. W najprostszym przypadku dla każdej iteracji pętli zewnętrznej pętla wewnętrzna wykonuje cały swój przebieg.

```
for (int row = 1; row <= 2; row++) {
    for (int col = 1; col <= 3; col++) {
        System.out.println(row + ":" + col);
    }
}
```

```
Wynik:
1:1
1:2
1:3
2:1
2:2
2:3
```

Najpierw `row` wynosi 1. Wewnętrzny licznik `col` przyjmuje 1, 2, 3. Po zakończeniu wewnętrznej pętli `row` rośnie do 2, a wewnętrzna pętla zaczyna się ponownie od `col = 1`. Łącznie wykonuje się 2 razy 3, czyli 6 wyświetleń. Taką liczbę powtórzeń często obliczymy przez pomnożenie liczby iteracji obu pętli.

10. Silnia obliczana przez iterację

Silnia liczby nieujemnej `n`, oznaczana `n!`, jest iloczynem liczb od 1 do `n`. Na przykład `4! = 1 * 2 * 3 * 4 = 24`. Z definicji `0! = 1`. W Javie można obliczyć silnię za pomocą pętli i akumulatora.

```
int n = 4;
long factorial = 1;
for (int i = 2; i <= n; i++) {
    factorial *= i;
}
System.out.println(factorial);
```

```
Wynik:
24
```

Przed pętlą akumulator wynosi 1. Po mnożeniu przez 2 ma wartość 2, po 3 wartość 6, po 4 wartość 24. Używamy `long`, bo silnia rośnie bardzo szybko, ale nawet ten typ ma ograniczony zakres: dla dużych `n` wynik przestanie się mieścić. Dla `n = 0` pętla nie wykona żadnej iteracji i poprawnie pozostanie 1. W praktyce trzeba także sprawdzić, czy użytkownik nie podał liczby ujemnej.

11. Rekurencja - metoda wywołuje samą siebie

Rekurencja jest innym sposobem rozwiązania problemu. W rekurencji metoda wywołuje samą siebie, zwykle z prostszym argumentem. Każde wywołanie ma własne parametry i zmienne lokalne. Koniecznie musi istnieć **przypadek bazowy**, czyli sytuacja, w której metoda zwraca wynik bez kolejnego wywołania siebie. Bez tego łatwo doprowadzić do błędu przepełnienia stosu (`StackOverflowError`).

```
static long factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

Dla argumentu 4 metoda potrzebuje `4 * factorial(3)`. Aby obliczyć `factorial(3)`, potrzebuje `3 * factorial(2)`, a następnie `2 * factorial(1)`. Ostatnie wywołanie trafia do przypadku bazowego i zwraca 1. Wyniki wracają kolejno: 2, 6, 24. Uwaga: przykład zakłada nieujemny argument. Dla wartości ujemnej należy osobno ustalić sposób obsługi błędnych danych zamiast niepostrzeżenie zwracać 1.

12. Ciąg Fibonacciego - przykład iteracji i rekurencji

Ciąg Fibonacciego w najczęściej spotykanej wersji rozpoczyna się od 0 i 1, a kolejny wyraz jest sumą dwóch poprzednich: 0, 1, 1, 2, 3, 5, 8, ... Możemy obliczać go iteracyjnie, pamiętając dwa ostatnie wyrazy, albo rekurencyjnie, korzystając ze wzoru matematycznego.

```
int a = 0;
int b = 1;
for (int i = 0; i < 7; i++) {
    System.out.print(a + " ");
    int next = a + b;
    a = b;
    b = next;
}
```

```
Wynik:
0 1 1 2 3 5 8
```

Po wyświetleniu aktualnego `a` obliczamy `next`, czyli sumę dwóch poprzednich wyrazów. Następnie przesuwamy wartości: dawne `b` staje się nowym `a`, a obliczona suma nowym `b`. Naiwna rekurencyjna implementacja Fibonacciego wielokrotnie oblicza te same wartości i dla większych danych jest znacznie wolniejsza. Rekurencja nie oznacza automatycznie lepszego rozwiązania.

Przykład 1. Example01While

Oglądamy pętlę, w której warunek jest sprawdzany przed każdą iteracją.

```
package block4;

public class Example01While {
    public static void main(String[] args) {
        int number = 1;
        while (number <= 5) {
            System.out.println(number);
            number++;
        }
    }
}
```

Wynik:

```
1
2
3
4
5
```

- Zapis `int number = 1;` tworzy zmienną `number` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Warunek `while (number <= 5) {` jest sprawdzany przy kolejnych przejściach pętli. Jeśli przestanie być prawdziwy, następna iteracja nie zostanie wykonana.
- Instrukcja `System.out.println(number);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `number++;` zmienia wartość istniejącej zmiennej. Jej nowa wartość będzie użyta przy następnym sprawdzeniu warunku lub obliczeniu.

Przykład 2. Example02DoWhile

Sprawdzamy, dlaczego `do-while` wykona blok co najmniej raz.

```
package block4;

public class Example02DoWhile {
    public static void main(String[] args) {
        int value = 10;
        do {
            System.out.println("Wartość: " + value);
            value++;
        } while (value < 10);
    }
}
```

Wynik:

```
Wartość: 10
```

- Zapis `int value = 10;` tworzy zmienną `value` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Instrukcja `System.out.println("Wartość: " + value);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `value++;` zmienia wartość istniejącej zmiennej. Jej nowa wartość będzie użyta przy następnym sprawdzeniu warunku lub obliczeniu.

Przykład 3. Example03For

Prześledzimy, jak rośnie licznik i kiedy pętla `for` kończy działanie.

```
package block4;

public class Example03For {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Iteracja " + i);
        }
    }
}
```

Wynik:

```
Iteracja 1
Iteracja 2
Iteracja 3
Iteracja 4
Iteracja 5
```

- Nagłówek `for (int i = 1; i <= 5; i++) {` określa wartość startową, warunek kontynuowania oraz sposób zmiany licznika. Trzeba sprawdzić, czy ostatnia iteracja mieści się w granicach.
- Instrukcja `System.out.println("Iteracja " + i);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 4. Example04BreakContinue

Na konkretnych liczbach porównujemy zakończenie całej pętli z pominięciem tylko jednej iteracji.

```
package block4;

public class Example04BreakContinue {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            if (i == 3) {
                continue;
            }
            if (i == 8) {
                break;
            }
            System.out.println(i);
        }
    }
}
```

Wynik:

```
1
2
4
5
6
7
```

- Nagłówek `for (int i = 1; i <= 10; i++) {` określa wartość startową, warunek kontynuowania oraz sposób zmiany licznika. Trzeba sprawdzić, czy ostatnia iteracja mieści

się w granicach.

- W zapisie `if (i == 3) {` program podejmuje decyzję na podstawie wartości logicznej. Instrukcje w bloku są wykonywane tylko wtedy, gdy wynik warunku jest `true`.
- W zapisie `if (i == 8) {` program podejmuje decyzję na podstawie wartości logicznej. Instrukcje w bloku są wykonywane tylko wtedy, gdy wynik warunku jest `true`.
- Instrukcja `System.out.println(i);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 5. Example05NestedLoops

Obserwujemy, jak pętla wewnętrzna zaczyna się od nowa dla kolejnych wartości licznika zewnętrznego.

```
package block4;

public class Example05NestedLoops {
    public static void main(String[] args) {
        for (int row = 1; row <= 3; row++) {
            for (int col = 1; col <= 4; col++) {
                System.out.print("(" + row + "," + col + ") ");
            }
            System.out.println();
        }
    }
}
```

Wynik:

```
(1,1) (1,2) (1,3) (1,4)
(2,1) (2,2) (2,3) (2,4)
(3,1) (3,2) (3,3) (3,4)
```

- Nagłówek `for (int row = 1; row <= 3; row++) {` określa wartość startową, warunek kontynuowania oraz sposób zmiany licznika. Trzeba sprawdzić, czy ostatnia iteracja mieści się w granicach.
- Nagłówek `for (int col = 1; col <= 4; col++) {` określa wartość startową, warunek kontynuowania oraz sposób zmiany licznika. Trzeba sprawdzić, czy ostatnia iteracja mieści się w granicach.
- Instrukcja `System.out.print("(" + row + "," + col + ") ");` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println();` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 6. Example06FactorialIterative

Obliczamy silnię w kolejnych powtórzeniach, mnożąc akumulator przez coraz większy czynnik.

```
package block4;

public class Example06FactorialIterative {
    public static void main(String[] args) {
        int n = 5;
        long result = 1;
        for (int i = 2; i <= n; i++) {
            result *= i;
        }
        System.out.println(result);
    }
}
```

Wynik:

```
120
```

- Zapis `int n = 5;` tworzy zmienną `n` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Zapis `long result = 1;` tworzy zmienną `result` typu `long` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Nagłówek `for (int i = 2; i <= n; i++) {` określa wartość startową, warunek kontynuowania oraz sposób zmiany licznika. Trzeba sprawdzić, czy ostatnia iteracja mieści się w granicach.
- Instrukcja `result *= i;` zmienia wartość istniejącej zmiennej. Jej nowa wartość będzie użyta przy następnym sprawdzeniu warunku lub obliczeniu.
- Instrukcja `System.out.println(result);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 7. Example07FactorialRecursive

Porównujemy obliczenie silni przez samowywołującą się metodę i przypadek bazowy.

```
package block4;

public class Example07FactorialRecursive {
    static long factorial(int n) {
        if (n <= 1) {
            return 1;
        }
        return n * factorial(n - 1);
    }

    public static void main(String[] args) {
        System.out.println(factorial(5));
    }
}
```

Wynik:

```
120
```

- W zapisie `if (n <= 1) {` program podejmuje decyzję na podstawie wartości logicznej. Instrukcje w bloku są wykonywane tylko wtedy, gdy wynik warunku jest `true`.
- Instrukcja `return 1;` kończy bieżące wywołanie metody i przekazuje wynik do miejsca wywołania.
- Instrukcja `return n * factorial(n - 1);` kończy bieżące wywołanie metody i przekazuje wynik do miejsca wywołania.
- Instrukcja `System.out.println(factorial(5));` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 8. Example08FibonacciIterative

Kolejne wyrazy ciągu otrzymujemy przez przesuwanie dwóch poprzednich wartości.

```
package block4;

public class Example08FibonacciIterative {
    public static void main(String[] args) {
        int count = 10;
        long a = 0;
        long b = 1;

        for (int i = 0; i < count; i++) {
            System.out.print(a + (i + 1 == count ? "\n" : " "));
            long next = a + b;
            a = b;
            b = next;
        }
    }
}
```

Wynik:

```
0 1 1 2 3 5 8 13 21 34
```

- Zapis `int count = 10;` tworzy zmienną `count` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Zapis `long a = 0;` tworzy zmienną `a` typu `long` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Zapis `long b = 1;` tworzy zmienną `b` typu `long` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Nagłówek `for (int i = 0; i < count; i++) {` określa wartość startową, warunek kontynuowania oraz sposób zmiany licznika. Trzeba sprawdzić, czy ostatnia iteracja mieści się w granicach.
- Instrukcja `System.out.print(a + (i + 1 == count ? "\n" : " "));` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Zapis `long next = a + b;` tworzy zmienną `next` typu `long` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.

Przykład 9. Example09FibonacciRecursive

Pokazujemy, że zwięzły zapis rekurencyjny może wykonywać wiele powtórzonych obliczeń.

```
package block4;

public class Example09FibonacciRecursive {
    static long fibonacci(int n) {
        if (n <= 1) {
            return n;
        }
        return fibonacci(n - 1) + fibonacci(n - 2);
    }

    public static void main(String[] args) {
        System.out.println(fibonacci(10));
    }
}
```

Wynik:

```
55
```

- W zapisie `if (n <= 1) {` program podejmuje decyzję na podstawie wartości logicznej. Instrukcje w bloku są wykonywane tylko wtedy, gdy wynik warunku jest `true`.
- Instrukcja `return n;` kończy bieżące wywołanie metody i przekazuje wynik do miejsca wywołania.
- Instrukcja `return fibonacci(n - 1) + fibonacci(n - 2);` kończy bieżące wywołanie metody i przekazuje wynik do miejsca wywołania.
- Instrukcja `System.out.println(fibonacci(10));` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

13. Zadania - pętle i wywołania rekurencyjne

Zadanie 1. Napisz pętlę `for`, która wypisze liczby od 10 do 1 malejąco. Sprawdź, czy liczby graniczne również zostają wypisane. Następnie napisz to samo za pomocą `while`.

Zadanie 2. Oblicz sumę liczb od 1 do 100. Wynik ma zostać obliczony przez pętlę, a nie wpisany ręcznie. Zapisz wartości akumulatora po pierwszych trzech iteracjach i po ostatniej.

Zadanie 3. Zmień przykład z `continue` tak, aby pomijał wszystkie wielokrotności liczby 3 od 1 do 12. Który fragment warunku odpowiada za wykrycie wielokrotności?

Zadanie 4. Narysuj ręcznie listę wywołań `factorial(5)` i wypisz wartości, które zwracają kolejne wywołania podczas cofania się do pierwszej metody.

Zadanie 5. Zbadaj, ile razy wykona się `println` w dwóch pętlach o zakresach `row = 0..3` i `col = 0..2`. Wyjaśnij, dlaczego wynik nie jest sumą 4 i 3.

Zadanie 6. Otwórz `Example09FibonacciRecursive` i `Example08FibonacciIterative`. Porównaj wyniki dla tego samego niewielkiego `n`. Nie zakładaj, że podobna odpowiedź oznacza identyczną liczbę wykonanych operacji.